Laboratorio 2
Ejercicio 1: Empresa de Muebles

1. R# claros y verificables.

R#	Nombre	Descripción	Criterio de verificación
R1	Configurar año de referencia	El sistema debe solicitar y capturar el año actual al inicio de la sesión para el cálculo temporal.	Se ingresa un valor entero positivo representativo de un año calendario (por ejemplo, 2026) y el sistema lo almacena en la empresa.
R2	Registrar datos del empleado	El sistema debe registrar nombre, documento, año de nacimiento, antigüedad, horas trabajadas y valor por hora para cada uno de los 3 empleados.	Se crean e inicializan 3 instancias de *Empleado* dentro de la *Empresa* con todos sus atributos asignados.
R3	Validar información del empleado	El sistema debe asegurar que las horas trabajadas, el valor de la hora y la antigüedad no sean negativos, y que el año de nacimiento sea menor o igual al año actual.	Si el usuario ingresa valores negativos en horas, tarifa o antigüedad, o un año de nacimiento mayor al actual o menor/igual a cero, el sistema rechaza la entrada, muestra el mensaje de error y solicita el dato nuevamente hasta que sea válido
R4	Calcular salario del empleado	El sistema debe calcular el salario de cada empleado multiplicando las horas trabajadas por el valor de la hora.	Con 40 horas y $10.000 por hora, el resultado devuelto es exactamente $400000.00
R5	Calcular edad del empleado	El sistema debe calcular la edad restando el año de nacimiento del año actual registrado.	Con año actual 2026 y año de nacimiento 1996, el valor devuelto es exactamente 30 años.
R6	Calcular nómina total	El sistema debe calcular la nómina global sumando los salarios individuales de los tres empleados.	Si los tres salarios son $400000.00, $300000.00 y $200000.00, la nómina total reportada es $900000.00
R7	Determinar empleado de mayor edad	El sistema debe comparar las edades de los tres trabajadores e identificar quién posee la mayor edad.	Si los empleados tienen 25, 45 y 30 años, el sistema reporta al empleado con 45 años.
R8	Presentar reporte general	El sistema debe imprimir en consola un reporte que incluya los datos de cada trabajador (nombre, documento, año de nacimiento, antigüedad, horas trabajadas y valor por hora) junto con su edad calculada y su salario. Además, el valor total de la nómina del periodo y el nombre y la edad del empleado de mayor edad.	Al solicitar el reporte, la consola muestra de manera estructurada y legible los 6 campos base más la edad y salario por cada empleado. Al final, se visualiza la nómina total con formato decimal ($XXXXXX.XX) y el nombre junto con la edad del empleado más anciano.

Requisitos NO Funcionales

R#	Nombre	Descripción	Criterio de verificación
RNF1	Estándar de compilación C++03	El código debe apegarse estrictamente a la especificación C++03.	Compila con banderas *-std=c++03 -Wall -Wextra -Werror -pedantic* sin warnings ni errores.
RNF2	Guía mínima de estilo de código	Sangría de 4 espacios, llaves estilo Allman, nomenclatura formal y comentarios Doxygen.	Se valida visualmente el checklist de estilo sin tabuladores y con llaves en su propio renglón.
RNF3	Modularidad y encapsulamiento	Separación de interfaces (.hpp) e implementaciones (.cpp), ocultamiento de atributos privados.	Existencia de *empleado.hpp*, *empleado.cpp*, *empresa.hpp*, *empresa.cpp*, *main.cpp* y *makefile*

2. Vocabulario

Término normalizado	Tipo	Definición	Sinonimos	¿Clase/ Atributo/ Operación candidata?
Empleado	Entidad	Persona contratada en el taller que produce bienes muebles.	Trabajador, operario	Clase
Empresa	Entidad	Organización "Muebles Modernos" que agrupa a los tres empleados.	Compañía	Clase
nombre_empleado	string	Cadena de texto con los nombres y apellidos del operario.	Identidad personal	Atributo
documento	string	Número de documento oficial de identidad del operario.	Cédula, ID	Atributo
anio_nacimiento	int	Año calendario en el que nació el empleado.	Fecha de nacimiento	Atributo
antiguedad	int	Cantidad de años completos que el empleado lleva vinculado.	Años trabajando	Atributo
horas_trabajadas	double	Número de horas laboradas durante el periodo de liquidación.	Tiempo trabajado	Atributo
valor_hora	double	Tarifa monetaria a pagar por cada hora trabajada.	Costo, tarifa	Atributo
anio_actual	int	Año en curso registrado por el administrador del taller.	Año presente	Atributo
CalcularSalario	Operación	Multiplica horas trabajadas por valor de la hora.	Calcular sueldo	Operación
CalcularEdad	Operación	Resta el año de nacimiento del año actual.	Obtener edad	Operación
CalcularNominaTotal	Operación	Suma los salarios individuales de los tres empleados.	Total nomina	Operación
ObtenerEmpleadoMayorEdad	Operación	Encuentra y entrega el empleado que posea mayor edad.	Buscar empleado más anciano	Operación
TOTAL_EMPLEADOS	int	Número fijo de trabajadores en la empresa (3).	Total trabajadores	Atributo
nombre_empresa	string	Nombre comercial de la organización ("Muebles Modernos").	Nombre taller	Atributo

3. CRC

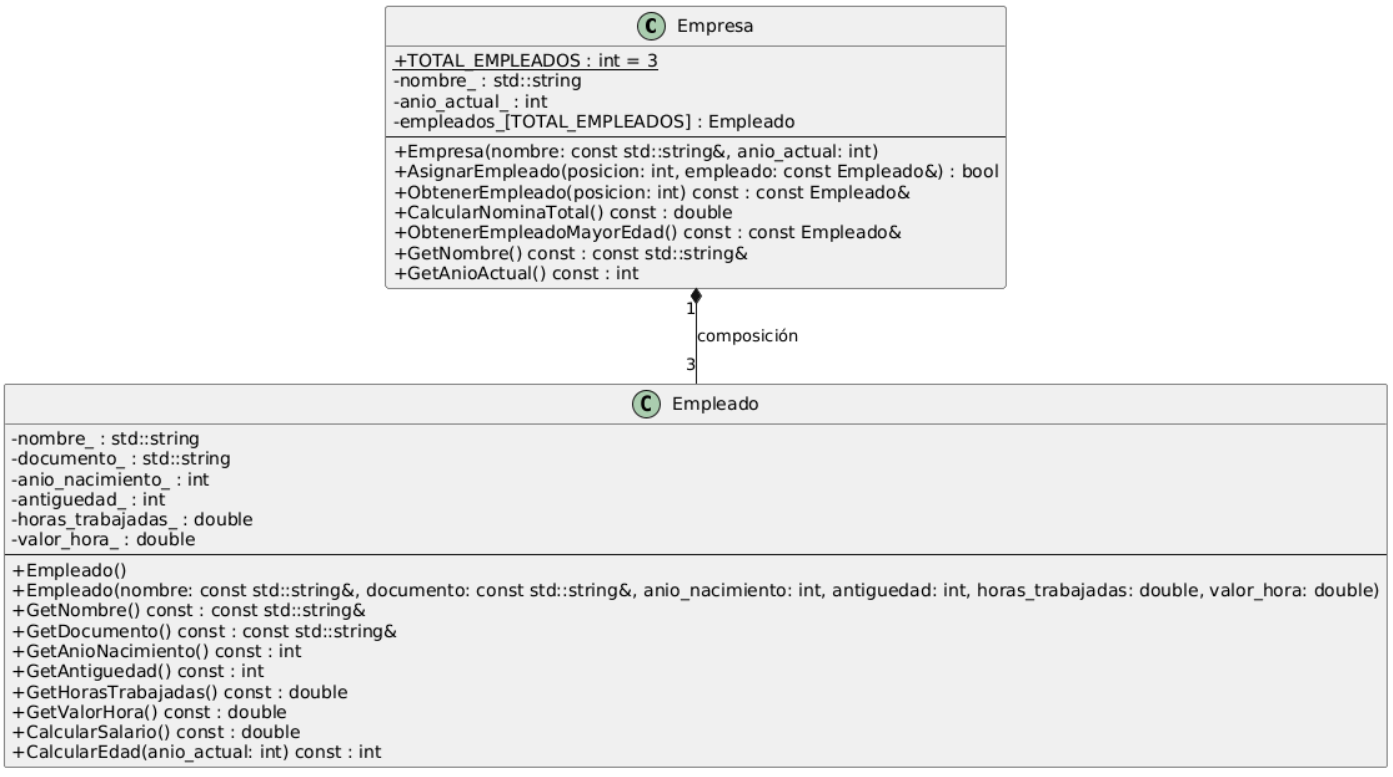
Campo	Contenido a completar
Clase	Empleado
Responsabilidades (qué hace)	1. Almacenar nombre, documento, año de nacimiento, antigüedad, horas trabajadas y valor por hora. 2. Proveer acceso de lectura (Getters) a nombre, documento, año de nacimiento, antigüedad, horas trabajadas y valor por hora para permitir la construcción del reporte detallado exigido en R8. 3. Calcular el salario devengado en el periodo (*horas_trabajadas_ * valor_hora*). 4. Calcular su edad en base al año actual *(anio_actual - anio_nacimiento*).
Colaboradores (a quién le pide)	Ninguna (Clase autónoma)
Mensajes clave (remitente → receptor.método(parámetros) → efecto/valor)	*empleado.CalcularSalario() -> double* *empleado.CalcularEdad(anio_actual: int) -> int*

Campo	Contenido a completar
Clase	Empresa
Responsabilidades (qué hace)	1. Almacenar el nombre de la empresa, el año actual y un arreglo fijo de 3 empleados. 2. Asignar y consultar empleados por posición. 3. Sumar los salarios de los empleados para obtener la nómina total del periodo. 4. Determinar cuál de los tres empleados tiene la mayor edad.
Colaboradores (a quién le pide)	Empleado (le solicita su salario y edad calculada)
Mensajes clave (remitente → receptor.método(parámetros) → efecto/valor)	*AsignarEmpleado(posicion: int, empleado: const Empleado&) -> bool* *ObtenerEmpleado(posicion: int) -> const Empleado&* *CalcularNominaTotal() -> double* *ObtenerEmpleadoMayorEdad() -> const Empleado&* *GetNombre() -> const std::string&*

Campo	Contenido a completar
Clase	Controlador (main)
Responsabilidades (qué hace)	1. Capturar el año de operación al inicio de la sesión (R1). 2. Gestionar la captura interactiva con reintentos para cada dato de los 3 empleados (R2, R3). 3. Instanciar los objetos Empleado y cargarlos en Empresa. 4. Presentar en consola el reporte general con todos los datos, la nómina total y el mayor de edad (R8).
Colaboradores (a quién le pide)	Empleado, Empresa
Mensajes clave (remitente → receptor.método(parámetros) → efecto/valor)	*taller.AsignarEmpleado(i, nuevo_empleado)* *taller.CalcularNominaTotal()* *taller.ObtenerEmpleadoMayorEdad()*

4. Diagrama de clase (UML)

Hecho en PlantUML (*https://www.planttext.com/*)



5. EPS/Contratos y 6. Pseudocódigo

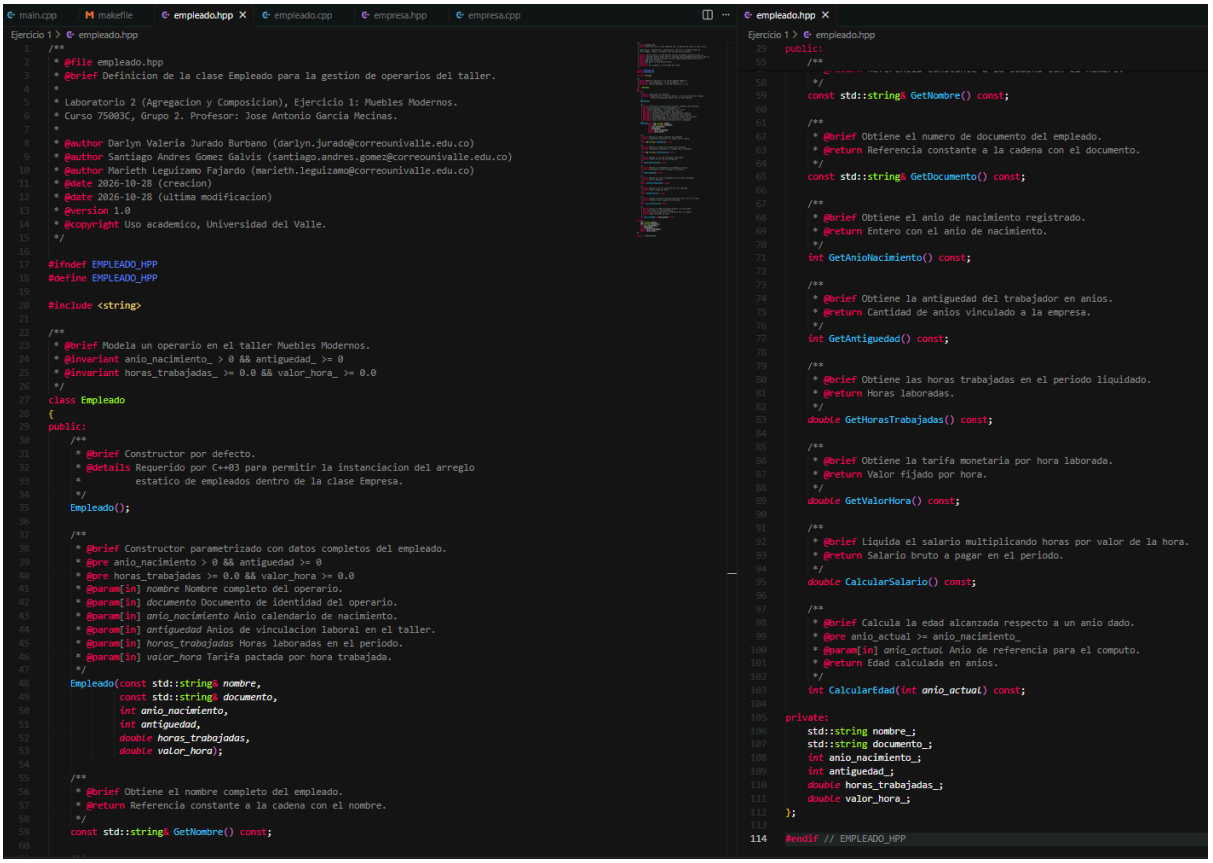
Campo	Contenido a contemplar
Clase	Empresa
Método	CalcularNominaTotal
Propósito (una línea)	Suma los salarios liquidados de todos los empleados del taller
E/P/S	
Entradas (datos requeridos)	Ninguna explícita.
Proceso (pasos, decisiones, fórmulas si aplica)	Inicializar acumulador en 0.0 y sumar el resultado de CalcularSalario() de cada empleado.
Salidas (resultado observable)	Nómina del periodo (double).
Contrato PRE/POST	
PRE	Arreglo de empleados inicializado.
POST	Retorna el total acumulado sin alterar los datos internos (const).
Pseudocódigo / Algoritmo	
Algoritmo	Algoritmo CalcularNominaTotal total_nomina ← 0.0 Para $i \leftarrow 0$ Hasta *TOTAL_EMPLEADOS - 1* Hacer total_nomina ← total_nomina + empleados_[i].CalcularSalario() FinPara Retornar total_nomina FinAlgoritmo

Campo	Contenido a contemplar
Clase	Empresa
Método	ObtenerEmpleadoMayorEdad
Propósito (una línea)	Identifica y retorna la referencia del operario que tiene la edad más alta en el taller.
E/P/S	
Entradas (datos requeridos)	Ninguna explícita (usa anio_actual_ y el arreglo empleados_).
Proceso (pasos, decisiones, fórmulas si aplica)	1. Fijar como mayor al primer empleado (posicion_mayor = 0) y calcular su edad. 2. Recorrer desde i = 1 hasta TOTAL_EMPLEADOS - 1 calculando la edad de cada operario. 3. Si la edad calculada es estrictamente mayor que la máxima actual (>), actualizar la máxima y la posición mayor (en caso de empate se conserva el primero). 4. Retornar la referencia al empleado en posicion_mayor.
Salidas (resultado observable)	Referencia constante al operario de mayor edad (const Empleado&).
Contrato PRE/POST	
PRE	anio_actual_ > 0 y los empleados del arreglo han sido inicializados
POST	Retorna el objeto del empleado cuya edad calculada es máxima; si hay empate gana el primero (comparación estricta >). No muta el estado (const).
Pseudocódigo / Algoritmo	
Algoritmo	Algoritmo ObtenerEmpleadoMayorEdad posicion_mayor ← 0 mayor_edad ← empleados_[0].CalcularEdad(anio_actual_) Para i ← 1 Hasta TOTAL_EMPLEADOS - 1 Hacer edad_actual ← empleados_[i].CalcularEdad(anio_actual_) Si edad_actual > mayor_edad Entonces mayor_edad ← edad_actual posicion_mayor ← i FinSi FinPara Retornar empleados_[posicion_mayor] FinAlgoritmo

Campo	Contenido a contemplar
Clase	Empresa
Método	ObtenerEmpleado
Propósito (una línea)	Retorna la referencia del empleado almacenado en la posición solicitada.
E/P/S	
Entradas (datos requeridos)	posicion (int).
Proceso (pasos, decisiones, fórmulas si aplica)	1. Comprobar que posicion esté dentro del rango $[0, TOTAL_EMPLEADOS - 1]$. 2. Si no es válido, lanzar una excepción de fuera de rango (std::out_of_range). 3. Retornar la referencia al elemento empleados_[posicion].
Salidas (resultado observable)	Referencia constante al empleado (const Empleado&).
Contrato PRE/POST	
PRE	0 <= posicion && posicion < TOTAL_EMPLEADOS
POST	Retorna la referencia al empleado sin hacer copias ni alterar el estado interno (const).
Pseudocódigo / Algoritmo	
Algoritmo	Algoritmo ObtenerEmpleado(posicion) Si posicion < 0 O posicion >= TOTAL_EMPLEADOS Entonces LanzarExcepcion ("Posicion fuera de rango") FinSi Retornar empleados_[posicion] FinAlgoritmo

6. Programa c++

empleado.hpp



```hpp
/**
 * @file empleado.hpp
 * @brief Definicion de la clase Empleado para la gestion de operarios del taller.
 *
 * Laboratorio 2 (Agregacion y Composicion), Ejercicio 1: Muebles Modernos.
 * Curso 75003C, Grupo 2. Profesor: Jose Antonio Garcia Mecinas.
 *
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @date 2026-10-28 (creacion)
 * @date 2026-10-28 (ultima modificacion)
 * @version 1.0
 * @copyright Uso academico, Universidad del Valle.
 */

#ifndef EMPLEADO_HPP
#define EMPLEADO_HPP

#include <string>

/**
 * @brief Modela un operario en el taller Muebles Modernos.
 * @invariant anio_nacimiento_ > 0 && antiguedad_ >= 0
 * @invariant horas_trabajadas_ >= 0.0 && valor_hora_ >= 0.0
 */
class Empleado
{
public:
    /**
     * @brief Constructor por defecto.
     * @details Requerido por C++03 para permitir la instanciacion del arreglo
     *          estatico de empleados dentro de la clase Empresa.
     */
    Empleado();

    /**
     * @brief Constructor parametrizado con datos completos del empleado.
     * @pre anio_nacimiento > 0 && antiguedad >= 0
     * @pre horas_trabajadas >= 0.0 && valor_hora >= 0.0
     * @param[in] nombre Nombre completo del operario.
     * @param[in] documento Documento de identidad del operario.
     * @param[in] anio_nacimiento Anio calendario de nacimiento.
     * @param[in] antiguedad Anios de vinculacion laboral en el taller.
     * @param[in] horas_trabajadas Horas laboradas en el periodo.
     * @param[in] valor_hora Tarifa pactada por hora trabajada.
     */
    Empleado(const std::string& nombre,
             const std::string& documento,
             int anio_nacimiento,
             int antiguedad,
             double horas_trabajadas,
             double valor_hora);

    /**
     * @brief Obtiene el nombre completo del empleado.
     * @return Referencia constante a la cadena con el nombre.
     */
    const std::string& GetNombre() const;

    /**
     * @brief Obtiene el numero de documento del empleado.
     * @return Referencia constante a la cadena con el documento.
     */
    const std::string& GetDocumento() const;

    /**
     * @brief Obtiene el anio de nacimiento registrado.
     * @return Entero con el anio de nacimiento.
     */
    int GetAnioNacimiento() const;

    /**
     * @brief Obtiene la antiguedad del trabajador en anios.
     * @return Cantidad de anios vinculado a la empresa.
     */
    int GetAntiguedad() const;

    /**
     * @brief Obtiene las horas trabajadas en el periodo liquidado.
     * @return Horas laboradas.
     */
    double GetHorasTrabajadas() const;

    /**
     * @brief Obtiene la tarifa monetaria por hora laborada.
     * @return Valor fijado por hora.
     */
    double GetValorHora() const;

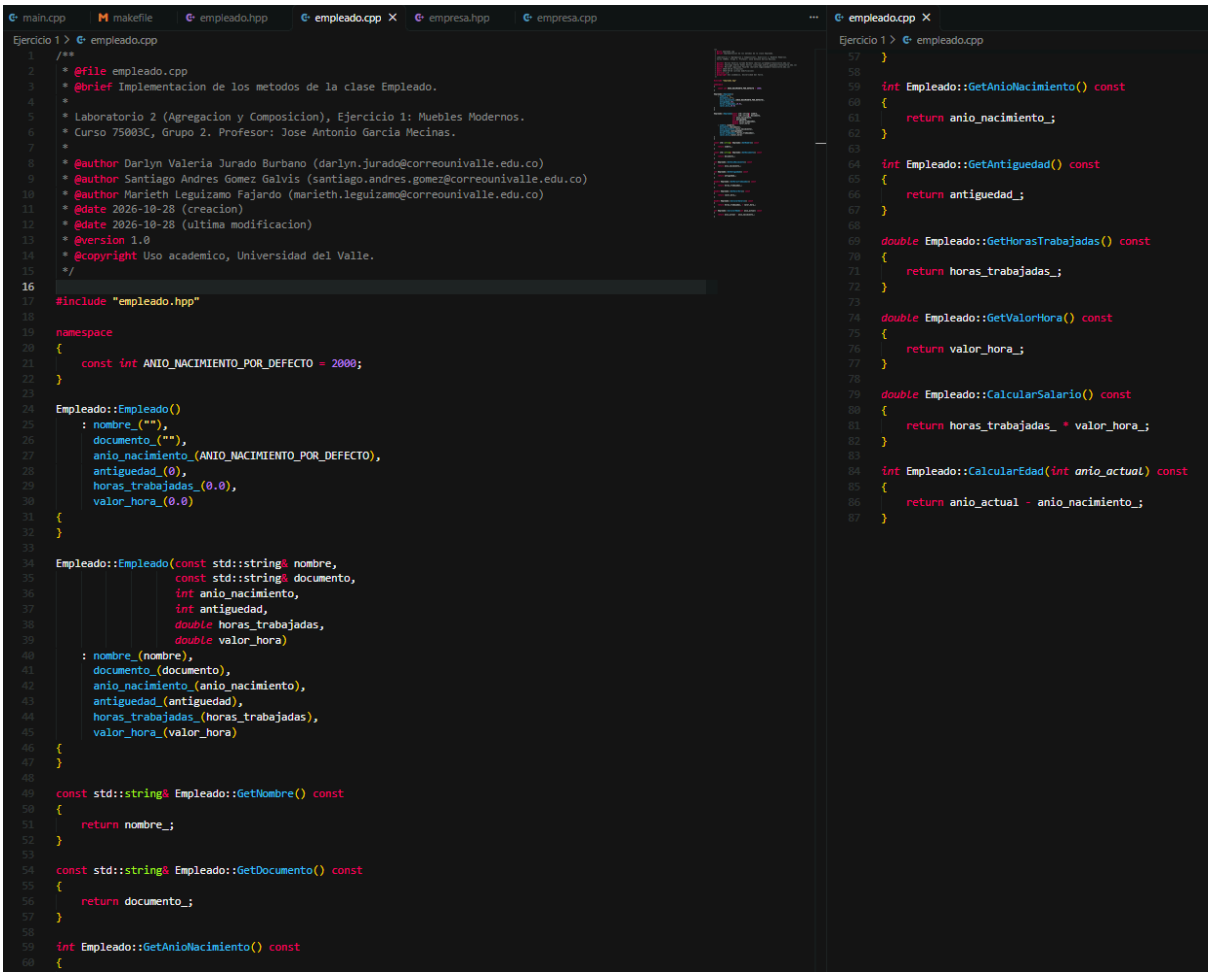
    /**
     * @brief Liquida el salario multiplicando horas por valor de la hora.
     * @return Salario bruto a pagar en el periodo.
     */
    double CalcularSalario() const;

    /**
     * @brief Calcula la edad alcanzada respecto a un anio dado.
     * @pre anio_actual >= anio_nacimiento_
     * @param[in] anio_actual Anio de referencia para el computo.
     * @return Edad calculada en anios.
     */
    int CalcularEdad(int anio_actual) const;

private:
    std::string nombre_;
    std::string documento_;
    int anio_nacimiento_;
    int antiguedad_;
    double horas_trabajadas_;
    double valor_hora_;
};

#endif // EMPLEADO_HPP
```

empleado.cpp



```cpp
/**
 * @file empleado.cpp
 * @brief Implementacion de los metodos de la clase Empleado.
 *
 * Laboratorio 2 (Agregacion y Composicion), Ejercicio 1: Muebles Modernos.
 * Curso 75003C, Grupo 2. Profesor: Jose Antonio Garcia Mecinas.
 *
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @date 2026-10-28 (creacion)
 * @date 2026-10-28 (ultima modificacion)
 * @version 1.0
 * @copyright Uso academico, Universidad del Valle.
 */

#include "empleado.hpp"

namespace
{
    const int ANIO_NACIMIENTO_POR_DEFECTO = 2000;
}

Empleado::Empleado()
    : nombre_(""),
      documento_(""),
      anio_nacimiento_(ANIO_NACIMIENTO_POR_DEFECTO),
      antiguedad_(0),
      horas_trabajadas_(0.0),
      valor_hora_(0.0)
{
}

Empleado::Empleado(const std::string& nombre,
                   const std::string& documento,
                   int anio_nacimiento,
                   int antiguedad,
                   double horas_trabajadas,
                   double valor_hora)
    : nombre_(nombre),
      documento_(documento),
      anio_nacimiento_(anio_nacimiento),
      antiguedad_(antiguedad),
      horas_trabajadas_(horas_trabajadas),
      valor_hora_(valor_hora)
{
}

const std::string& Empleado::GetNombre() const
{
    return nombre_;
}

const std::string& Empleado::GetDocumento() const
{
    return documento_;
}

int Empleado::GetAnioNacimiento() const
{
    return anio_nacimiento_;
}

int Empleado::GetAntiguedad() const
{
    return antiguedad_;
}

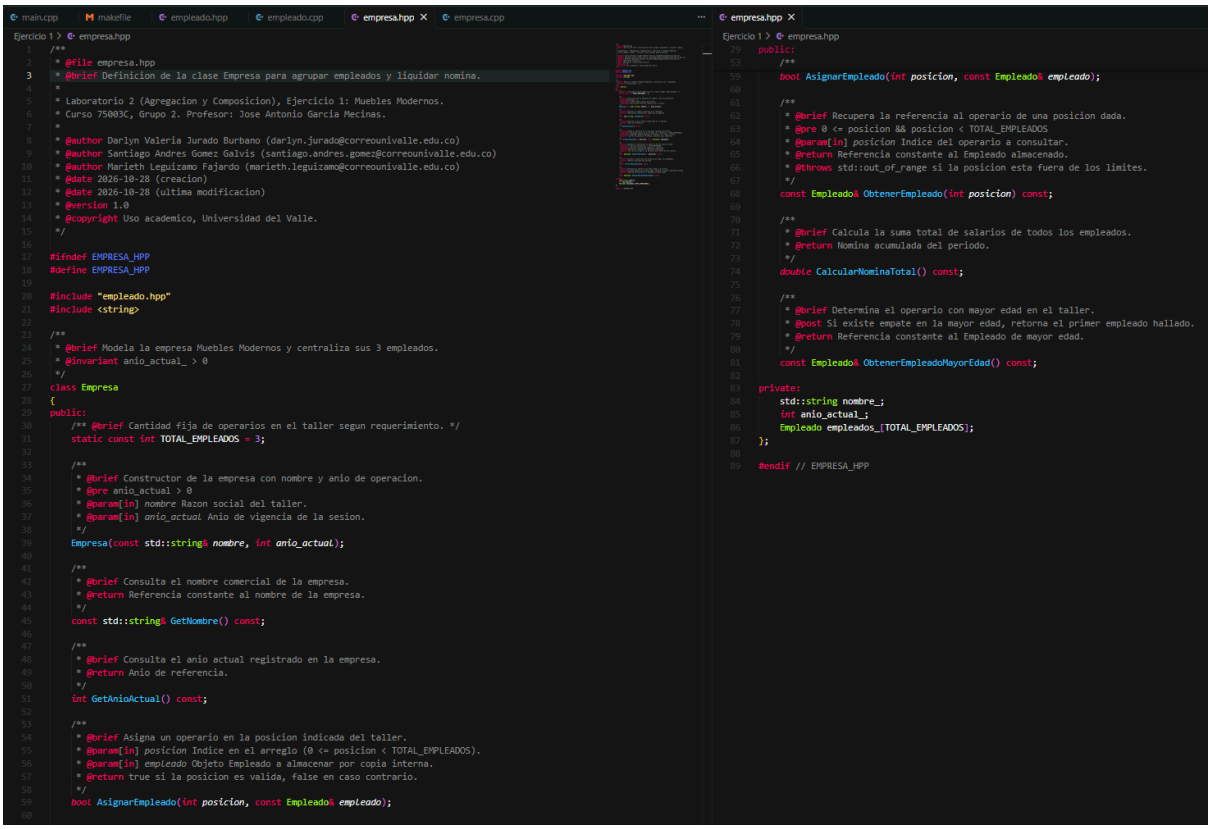
double Empleado::GetHorasTrabajadas() const
{
    return horas_trabajadas_;
}

double Empleado::GetValorHora() const
{
    return valor_hora_;
}

double Empleado::CalcularSalario() const
{
    return horas_trabajadas_ * valor_hora_;
}

int Empleado::CalcularEdad(int anio_actual) const
{
    return anio_actual - anio_nacimiento_;
}
```

empresa.hpp



```hpp
/**
 * @file empresa.hpp
 * @brief Definicion de la clase Empresa para agrupar empleados y liquidar nomina.
 *
 * Laboratorio 2 (Agregacion y Composicion), Ejercicio 1: Muebles Modernos.
 * Curso 75003C, Grupo 2. Profesor: Jose Antonio Garcia Mecinas.
 *
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @date 2026-10-28 (creacion)
 * @date 2026-10-28 (ultima modificacion)
 * @version 1.0
 * @copyright Uso academico, Universidad del Valle.
 */

#ifndef EMPRESA_HPP
#define EMPRESA_HPP

#include "empleado.hpp"
#include <string>

/**
 * @brief Modela la empresa Muebles Modernos y centraliza sus 3 empleados.
 * @invariant anio_actual_ > 0
 */
class Empresa
{
public:
    /** @brief Cantidad fija de operarios en el taller segun requerimiento. */
    static const int TOTAL_EMPLEADOS = 3;

    /**
     * @brief Constructor de la empresa con nombre y anio de operacion.
     * @pre anio_actual > 0
     * @param[in] nombre Razon social del taller.
     * @param[in] anio_actual Anio de vigencia de la sesion.
     */
    Empresa(const std::string& nombre, int anio_actual);

    /**
     * @brief Consulta el nombre comercial de la empresa.
     * @return Referencia constante al nombre de la empresa.
     */
    const std::string& GetNombre() const;

    /**
     * @brief Consulta el anio actual registrado en la empresa.
     * @return Anio de referencia.
     */
    int GetAnioActual() const;

    /**
     * @brief Asigna un operario en la posicion indicada del taller.
     * @param[in] posicion Indice en el arreglo (0 <= posicion < TOTAL_EMPLEADOS).
     * @param[in] empleado Objeto Empleado a almacenar por copia interna.
     * @return true si la posicion es valida, false en caso contrario.
     */
    bool AsignarEmpleado(int posicion, const Empleado& empleado);

    /**
     * @brief Recupera la referencia al operario de una posicion dada.
     * @pre 0 <= posicion && posicion < TOTAL_EMPLEADOS
     * @param[in] posicion Indice del operario a consultar.
     * @return Referencia constante al Empleado almacenado.
     * @throws std::out_of_range si la posicion esta fuera de los limites.
     */
    const Empleado& ObtenerEmpleado(int posicion) const;

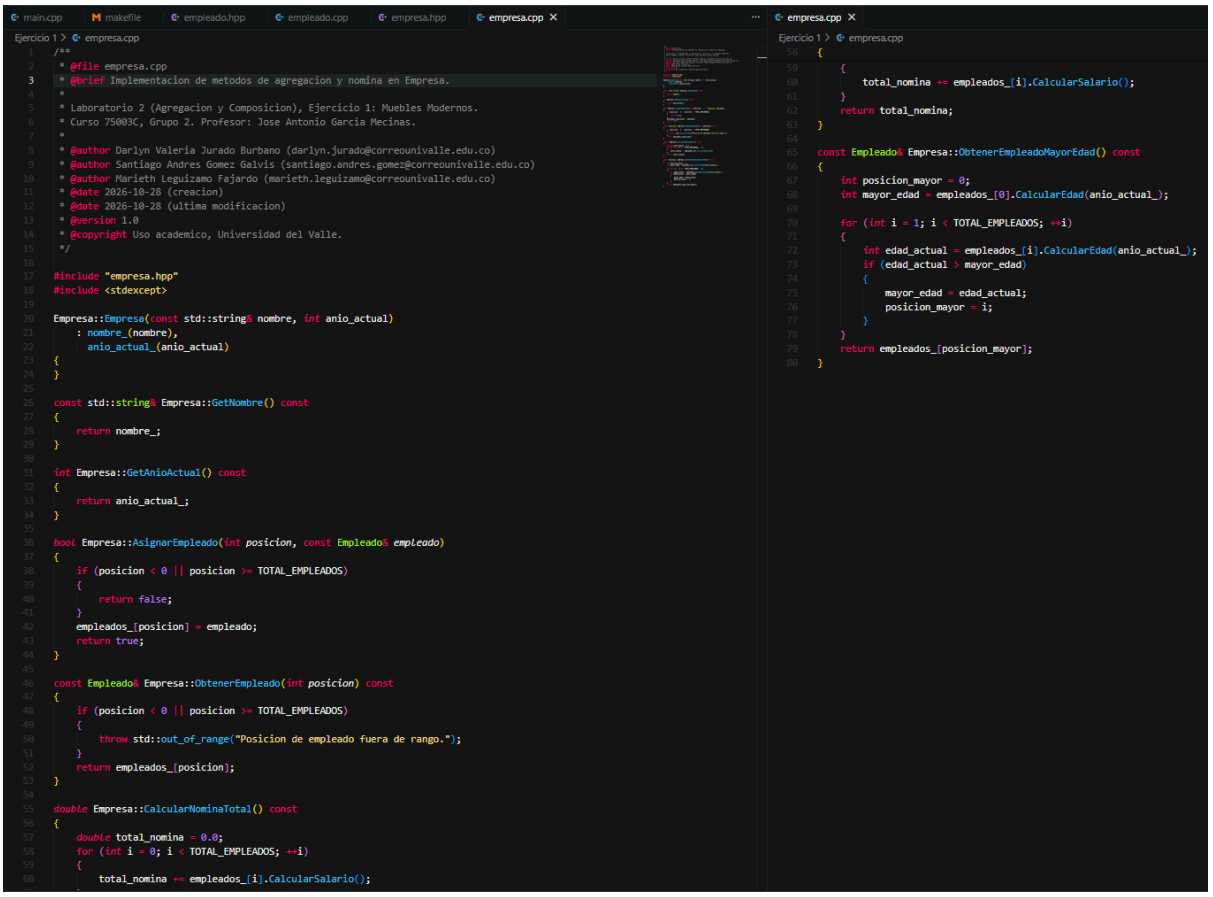
    /**
     * @brief Calcula la suma total de salarios de todos los empleados.
     * @return Nomina acumulada del periodo.
     */
    double CalcularNominaTotal() const;

    /**
     * @brief Determina el operario con mayor edad en el taller.
     * @post Si existe empate en la mayor edad, retorna el primer empleado hallado.
     * @return Referencia constante al Empleado de mayor edad.
     */
    const Empleado& ObtenerEmpleadoMayorEdad() const;

private:
    std::string nombre_;
    int anio_actual_;
    Empleado empleados_[TOTAL_EMPLEADOS];
};

#endif // EMPRESA_HPP
```

empresa.cpp



```cpp
/**
 * @file empresa.cpp
 * @brief Implementacion de metodos de agregacion y nomina en Empresa.
 *
 * Laboratorio 2 (Agregacion y Composicion), Ejercicio 1: Muebles Modernos.
 * Curso 75003C, Grupo 2. Profesor: Jose Antonio Garcia Mecinas.
 *
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @date 2026-10-28 (creacion)
 * @date 2026-10-28 (ultima modificacion)
 * @version 1.0
 * @copyright Uso academico, Universidad del Valle.
 */

#include "empresa.hpp"
#include <stdexcept>

Empresa::Empresa(const std::string& nombre, int anio_actual)
    : nombre_(nombre),
      anio_actual_(anio_actual)
{
}

const std::string& Empresa::GetNombre() const
{
    return nombre_;
}

int Empresa::GetAnioActual() const
{
    return anio_actual_;
}

bool Empresa::AsignarEmpleado(int posicion, const Empleado& empleado)
{
    if (posicion < 0 || posicion >= TOTAL_EMPLEADOS)
    {
        return false;
    }
    empleados_[posicion] = empleado;
    return true;
}

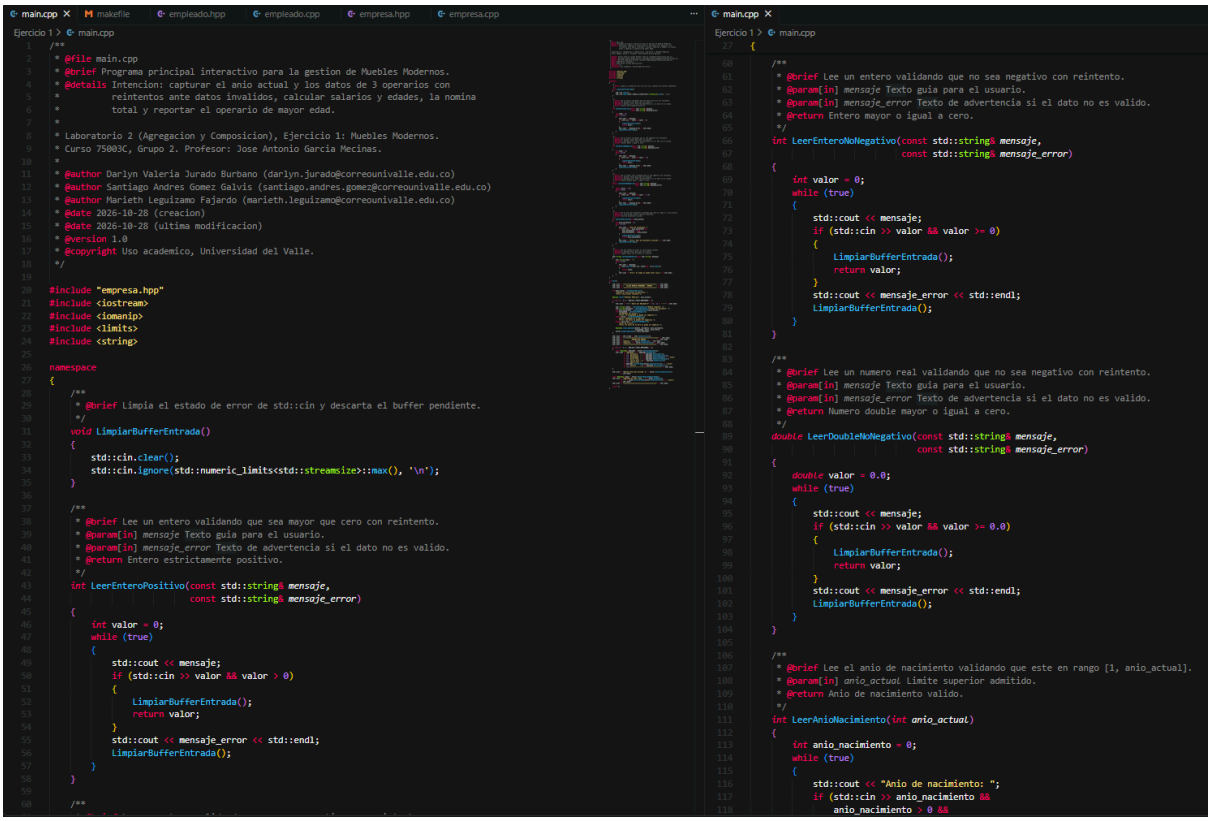
const Empleado& Empresa::ObtenerEmpleado(int posicion) const
{
    if (posicion < 0 || posicion >= TOTAL_EMPLEADOS)
    {
        throw std::out_of_range("Posicion de empleado fuera de rango.");
    }
    return empleados_[posicion];
}

double Empresa::CalcularNominaTotal() const
{
    double total_nomina = 0.0;
    for (int i = 0; i < TOTAL_EMPLEADOS; ++i)
    {
        total_nomina += empleados_[i].CalcularSalario();
    }
    return total_nomina;
}

const Empleado& Empresa::ObtenerEmpleadoMayorEdad() const
{
    int posicion_mayor = 0;
    int mayor_edad = empleados_[0].CalcularEdad(anio_actual_);

    for (int i = 1; i < TOTAL_EMPLEADOS; ++i)
    {
        int edad_actual = empleados_[i].CalcularEdad(anio_actual_);
        if (edad_actual > mayor_edad)
        {
            mayor_edad = edad_actual;
            posicion_mayor = i;
        }
    }
    return empleados_[posicion_mayor];
}
```

main.cpp



```cpp
/**
 * @file main.cpp
 * @brief Programa principal interactivo para la gestion de Muebles Modernos.
 * @details Intencion: capturar el anio actual y los datos de 3 operarios con
 *          reintentos ante datos invalidos, calcular salarios y edades, la nomina
 *          total y reportar el operario de mayor edad.
 *
 * Laboratorio 2 (Agregacion y Composicion), Ejercicio 1: Muebles Modernos.
 * Curso 75003C, Grupo 2. Profesor: Jose Antonio Garcia Mecinas.
 *
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @date 2026-10-28 (creacion)
 * @date 2026-10-28 (ultima modificacion)
 * @version 1.0
 * @copyright Uso academico, Universidad del Valle.
 */

#include "empresa.hpp"
#include <iostream>
#include <iomanip>
#include <limits>
#include <string>

namespace
{
    /**
     * @brief Limpia el estado de error de std::cin y descarta el buffer pendiente.
     */
    void LimpiarBufferEntrada()
    {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }

    /**
     * @brief Lee un entero validando que sea mayor que cero con reintento.
     * @param[in] mensaje Texto guia para el usuario.
     * @param[in] mensaje_error Texto de advertencia si el dato no es valido.
     * @return Entero estrictamente positivo.
     */
    int LeerEnteroPositivo(const std::string& mensaje,
                           const std::string& mensaje_error)
    {
        int valor = 0;
        while (true)
        {
            std::cout << mensaje;
            if (std::cin >> valor && valor > 0)
            {
                LimpiarBufferEntrada();
                return valor;
            }
            std::cout << mensaje_error << std::endl;
            LimpiarBufferEntrada();
        }
    }

    /**
     * @brief Lee un entero validando que no sea negativo con reintento.
     * @param[in] mensaje Texto guia para el usuario.
     * @param[in] mensaje_error Texto de advertencia si el dato no es valido.
     * @return Entero mayor o igual a cero.
     */
    int LeerEnteroNoNegativo(const std::string& mensaje,
                             const std::string& mensaje_error)
    {
        int valor = 0;
        while (true)
        {
            std::cout << mensaje;
            if (std::cin >> valor && valor >= 0)
            {
                LimpiarBufferEntrada();
                return valor;
            }
            std::cout << mensaje_error << std::endl;
            LimpiarBufferEntrada();
        }
    }

    /**
     * @brief Lee un numero real validando que no sea negativo con reintento.
     * @param[in] mensaje Texto guia para el usuario.
     * @param[in] mensaje_error Texto de advertencia si el dato no es valido.
     * @return Numero double mayor o igual a cero.
     */
    double LeerDoubleNoNegativo(const std::string& mensaje,
                                const std::string& mensaje_error)
    {
        double valor = 0.0;
        while (true)
        {
            std::cout << mensaje;
            if (std::cin >> valor && valor >= 0.0)
            {
                LimpiarBufferEntrada();
                return valor;
            }
            std::cout << mensaje_error << std::endl;
            LimpiarBufferEntrada();
        }
    }

    /**
     * @brief Lee el anio de nacimiento validando que este en rango [1, anio_actual].
     * @param[in] anio_actual Limite superior admitido.
     * @return Anio de nacimiento valido.
     */
    int LeerAnioNacimiento(int anio_actual)
    {
        int anio_nacimiento = 0;
        while (true)
        {
            std::cout << "Anio de nacimiento: ";
            if (std::cin >> anio_nacimiento &&
                anio_nacimiento > 0 &&
                anio_nacimiento <= anio_actual)
            {
                LimpiarBufferEntrada();
                return anio_nacimiento;
            }
            std::cout << "Error: Anio de nacimiento invalido." << std::endl;
            LimpiarBufferEntrada();
        }
    }

    /**
     * @brief Lee una cadena de texto no vacia desde consola.
     * @param[in] mensaje Texto guia para el usuario.
     * @return Cadena leida con al menos un caracter.
     */
    std::string LeerCadenaNoVacia(const std::string& mensaje)
    {
        std::string texto = "";
        while (true)
        {
            std::cout << mensaje;
            if (std::getline(std::cin, texto) && !texto.empty())
            {
                return texto;
            }
            std::cout << "Error: El campo no puede estar vacio." << std::endl;
        }
    }
}

int main()
{
    std::cout << "======================================" << std::endl;
    std::cout << "   TALLER MUEBLES MODERNOS - NOMINA   " << std::endl;
    std::cout << "======================================" << std::endl;

    int anio_actual = LeerEnteroPositivo(
        "Ingrese el anio actual de operacion: ",
        "Error: Anio actual invalido.");

    Empresa taller("Muebles Modernos", anio_actual);

    for (int i = 0; i < Empresa::TOTAL_EMPLEADOS; ++i)
    {
        std::cout << "\n--- Datos del Empleado #" << (i + 1) << " ---" << std::endl;

        std::string nombre = LeerCadenaNoVacia("Nombre completo: ");
        std::string documento = LeerCadenaNoVacia("Numero de documento: ");
        int anio_nacimiento = LeerAnioNacimiento(anio_actual);
        int antiguedad = LeerEnteroNoNegativo(
            "Antiguedad (en anios): ",
            "Error: La antiguedad no puede ser negativa.");
        double horas = LeerDoubleNoNegativo(
            "Horas trabajadas en el periodo: ",
            "Error: Las horas no pueden ser negativas.");
        double valor_hora = LeerDoubleNoNegativo(
            "Valor de la hora: ",
            "Error: El valor de la hora no puede ser negativo.");

        Empleado nuevo_empleado(nombre, documento, anio_nacimiento,
                                antiguedad, horas, valor_hora);
        taller.AsignarEmpleado(i, nuevo_empleado);
    }

    std::cout << std::fixed << std::setprecision(2);
    std::cout << "\n======================================" << std::endl;
    std::cout << "          REPORTE DE NOMINA           " << std::endl;
    std::cout << "Empresa: " << taller.GetNombre() << std::endl;
    std::cout << "Vigencia: " << taller.GetAnioActual() << std::endl;
    std::cout << "======================================" << std::endl;

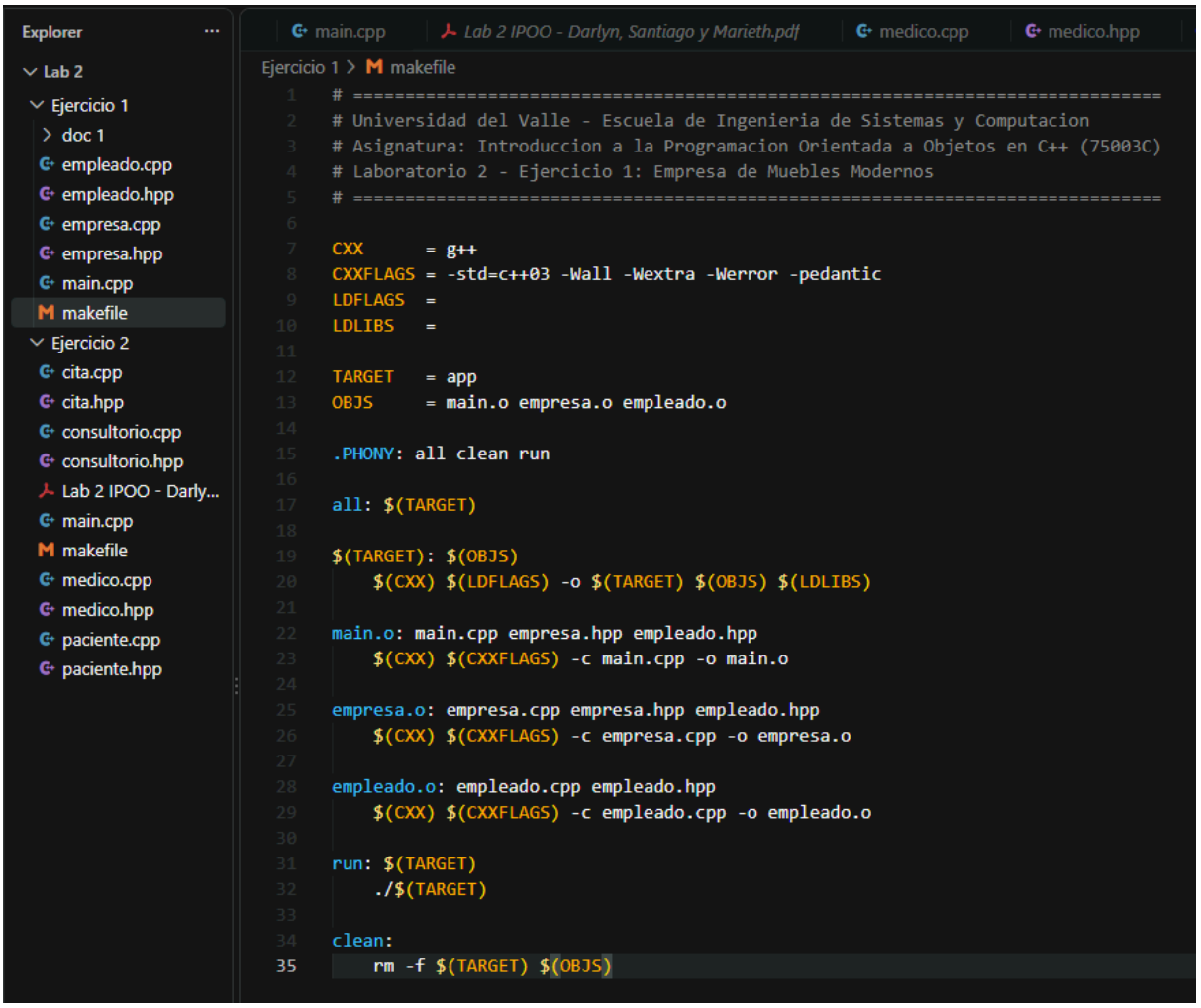
    for (int j = 0; j < Empresa::TOTAL_EMPLEADOS; ++j)
    {
        const Empleado& empleado = taller.ObtenerEmpleado(j);
        std::cout << "Empleado: " << empleado.GetNombre()
                  << "\n  Documento   : " << empleado.GetDocumento()
                  << "\n  Nacimiento  : " << empleado.GetAnioNacimiento()
                  << "\n  Antiguedad  : " << empleado.GetAntiguedad() << " anios"
                  << "\n  Horas trab. : " << empleado.GetHorasTrabajadas()
                  << "\n  Tarifa hora : $" << empleado.GetValorHora()
                  << "\n  Edad actual : "
                  << empleado.CalcularEdad(taller.GetAnioActual()) << " anios"
                  << "\n  Salario     : $" << empleado.CalcularSalario()
                  << "\n----------------------------------------" << std::endl;
    }

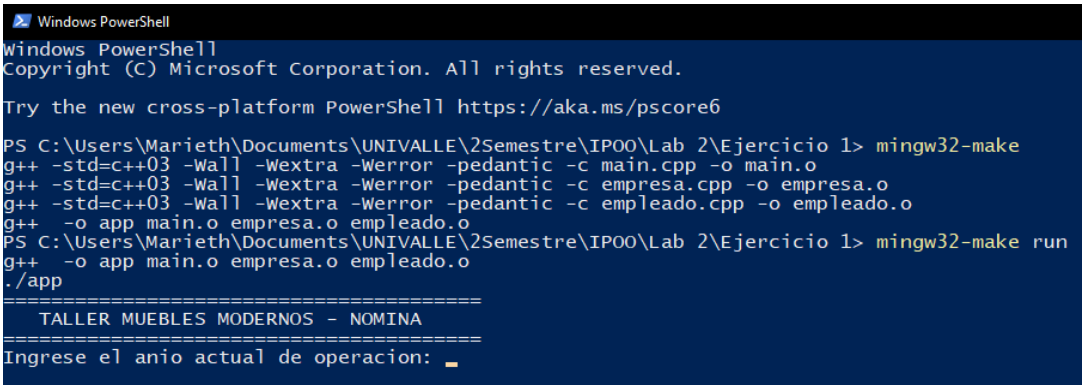
    std::cout << "Nomina total del periodo: $" << taller.CalcularNominaTotal()
              << std::endl;

    const Empleado& mayor = taller.ObtenerEmpleadoMayorEdad();
    std::cout << "Empleado de mayor edad  : " << mayor.GetNombre()
              << " (" << mayor.CalcularEdad(taller.GetAnioActual()) << " anios)"
              << std::endl;
    std::cout << "======================================" << std::endl;

    return 0;
}
```

makefile

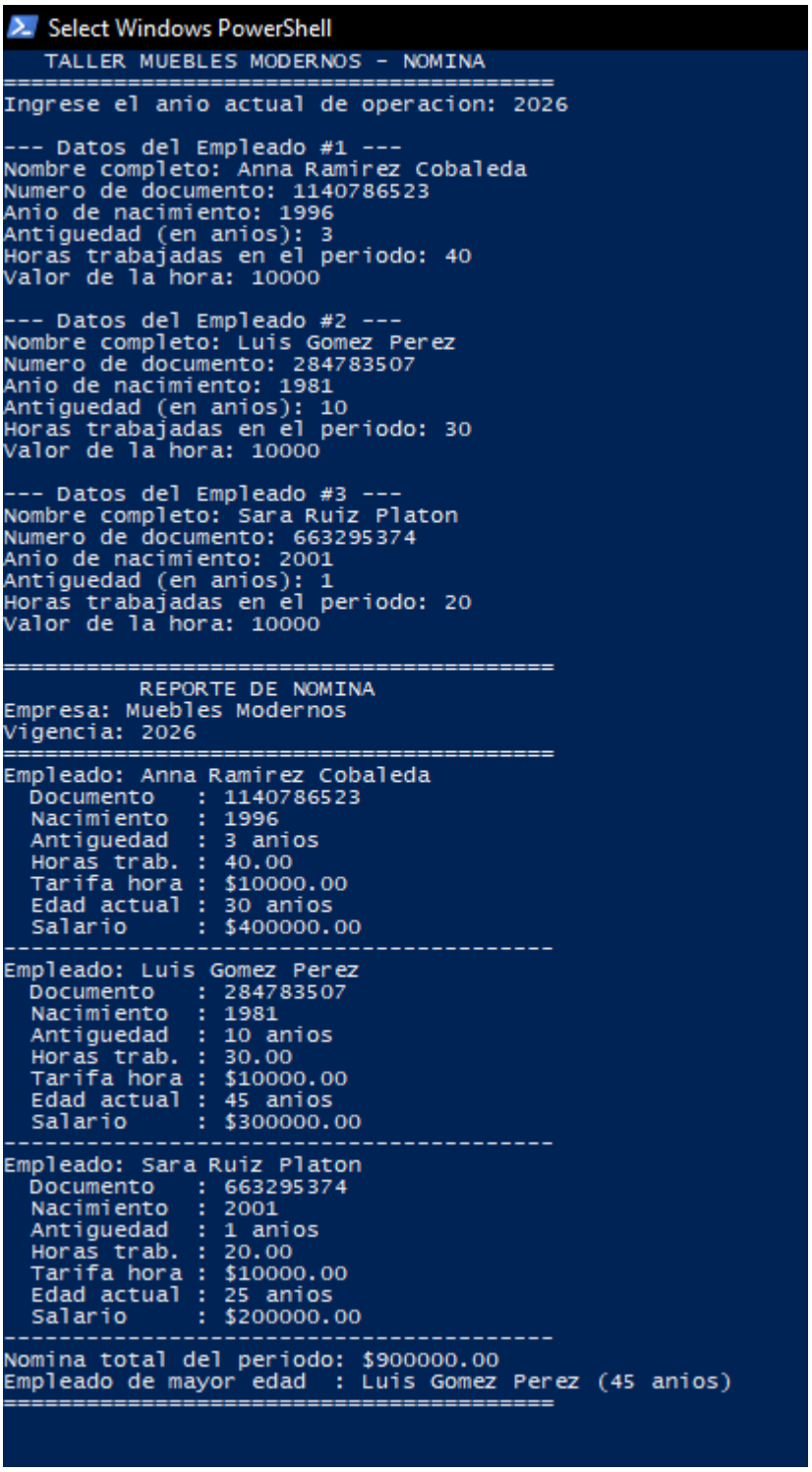

Explorer
Lab 2
Ejercicio 1
doc 1
empleado.cpp
empleado.hpp
empresa.cpp
empresa.hpp
main.cpp
makefile
Ejercicio 2
cita.cpp
cita.hpp
consultorio.cpp
consultorio.hpp
Lab 2 IPOO - Darly...
main.cpp
makefile
medico.cpp
medico.hpp
paciente.cpp
paciente.hpp
main.cpp
Lab 2 IPOO - Darlyn, Santiago y Marieth.pdf
medico.cpp
medico.hpp
Ejercicio 1 > makefile
==
Universidad del Valle - Escuela de Ingenieria de Sistemas y Computacion
Asignatura: Introduccion a la Programacion Orientada a Objetos en C++ (75003C)
Laboratorio 2 - Ejercicio 1: Empresa de Muebles Modernos
==
CXX = g++
CXXFLAGS = -std=c++03 -Wall -Wextra -Werror -pedantic
LDFLAGS =
LDLIBS =
TARGET = app
OBJS = main.o empresa.o empleado.o
.PHONY: all clean run
all: $(TARGET)
$(TARGET): $(OBJS)
$(CXX) $(LDFLAGS) -o $(TARGET) $(OBJS) $(LDLIBS)
main.o: main.cpp empresa.hpp empleado.hpp
$(CXX) $(CXXFLAGS) -c main.cpp -o main.o
empresa.o: empresa.cpp empresa.hpp empleado.hpp
$(CXX) $(CXXFLAGS) -c empresa.cpp -o empresa.o
empleado.o: empleado.cpp empleado.hpp
$(CXX) $(CXXFLAGS) -c empleado.cpp -o empleado.o
run: $(TARGET)
./$(TARGET)
clean:
rm -f $(TARGET) $(OBJS)


Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Try the new cross-platform PowerShell https://aka.ms/pscore6
PS C:\Users\Marieth\Documents\UNIVALLE\2Semestre\IPOO\Lab 2\Ejercicio 1> mingw32-make
g++ -std=c++03 -Wall -Wextra -Werror -pedantic -c main.cpp -o main.o
g++ -std=c++03 -Wall -Wextra -Werror -pedantic -c empresa.cpp -o empresa.o
g++ -std=c++03 -Wall -Wextra -Werror -pedantic -c empleado.cpp -o empleado.o
g++ -o app main.o empresa.o empleado.o
PS C:\Users\Marieth\Documents\UNIVALLE\2Semestre\IPOO\Lab 2\Ejercicio 1> mingw32-make run
g++ -o app main.o empresa.o empleado.o
./app
TALLER MUEBLES MODERNOS - NOMINA
Ingrese el anio actual de operacion:

7. Evidencias

Casos válidos

Caso base (R4 a R8)



Empate de edades

```
Windows PowerShell
./app
========================================
   TALLER MUEBLES MODERNOS - NOMINA
========================================
Ingrese el anio actual de operacion: 2026

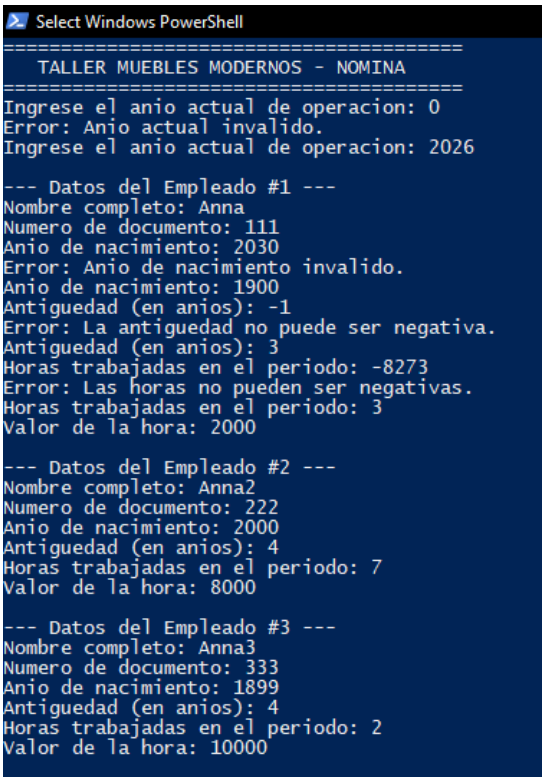
--- Datos del Empleado #1 ---
Nombre completo: Anna
Numero de documento: 111
Anio de nacimiento: 1990
Antiguedad (en anios): 5
Horas trabajadas en el periodo: 10
Valor de la hora: 5000

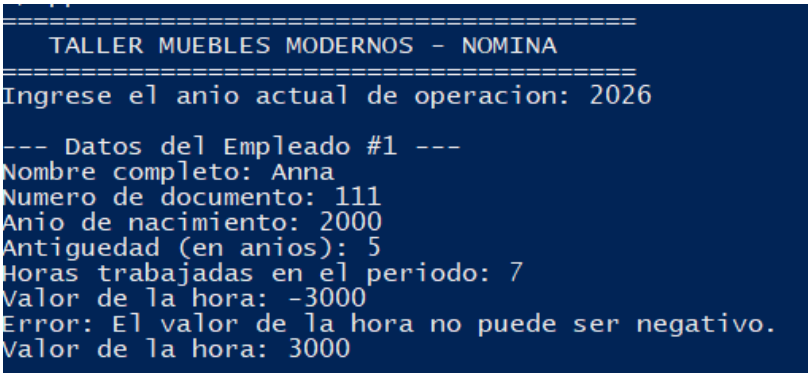
--- Datos del Empleado #2 ---
Nombre completo: Juan
Numero de documento: 222
Anio de nacimiento: 1990
Antiguedad (en anios): 5
Horas trabajadas en el periodo: 10
Valor de la hora: 5000

--- Datos del Empleado #3 ---
Nombre completo: Carlos
Numero de documento: 333
Anio de nacimiento: 2026
Antiguedad (en anios): 5
Horas trabajadas en el periodo: 10
Valor de la hora: 5000

========================================
          REPORTE DE NOMINA
Empresa: Muebles Modernos
Vigencia: 2026
========================================
Empleado: Anna
  Documento   : 111
  Nacimiento  : 1990
  Antiguedad  : 5 anios
  Horas trab. : 10.00
  Tarifa hora : $5000.00
  Edad actual : 36 anios
  Salario     : $50000.00
----------------------------------------
Empleado: Juan
  Documento   : 222
  Nacimiento  : 1990
  Antiguedad  : 5 anios
  Horas trab. : 10.00
  Tarifa hora : $5000.00
  Edad actual : 36 anios
  Salario     : $50000.00
----------------------------------------
Empleado: Carlos
  Documento   : 333
  Nacimiento  : 2026
  Antiguedad  : 5 anios
  Horas trab. : 10.00
  Tarifa hora : $5000.00
  Edad actual : 0 anios
  Salario     : $50000.00
----------------------------------------
Nomina total del periodo: $150000.00
Empleado de mayor edad  : Anna (36 anios)
========================================
PS C:\Users\Marieth\Documents\UNIVALLE\2Semestre\IPOO\Lab 2\Ejercicio 1>
```

Casos inválidos
Entradas inválidas con reintento





(Error de valor de hora)

(Errores de año actual, nacimiento, antigüedad y horas)

